Laboratory Work No. 2

Part 1. Normalization

Task 1. Analysis of the Initial Table

Initial table:

```
Booking_receipt(booking_id, passenger_full_name, passenger_passport_number,
flight_number, departure_airport_name, departure_city, arrival_airport_name,
arrival_city, airline_name, seat_numbers, ticket_price)
```

The seat_numbers field contains several values in one cell, for example "12A, 12B". Therefore, the initial table does not satisfy 1NF.

After conversion to 1NF, the composite key (booking_id, seat_number) is formed. Some non-key attributes are determined only by booking_id rather than by the whole composite key. Therefore, the structure after 1NF does not satisfy 2NF.

The task statement gives the dependencies booking_id → flight_number and flight_number → airline_name. They form a transitive dependency, so the structure after 2NF must be converted to 3NF.

Task 2. Step-by-Step Normalization of Booking_receipt

Initial Table

Field	Description
booking_id	Booking identifier
passenger_full_name	Passenger full name
passenger_passport_number	Passenger passport number
flight_number	Flight number
departure_airport_name	Departure airport
departure_city	Departure city
arrival_airport_name	Arrival airport
arrival_city	Arrival city
airline_name	Airline name
seat_numbers	List of seats, for example "12A, 12B"
ticket_price	Ticket price

1NF

Why the initial table does not satisfy 1NF

The seat_numbers field stores several values at the same time. For example: seat_numbers = "12A, 12B". In 1NF, each field must contain one atomic value.

Transformation

The seat_numbers values were split into separate rows, and the field was represented as seat_number.

booking_id	passenger_full_name	flight_number	seat_number	ticket_price
1	Passenger A	KC101	12A	45000
1	Passenger A	KC101	12B	45000

Structure after 1NF

Field	Key
booking_id	PK (part)
seat_number	PK (part)
passenger_full_name	
passenger_passport_number	
flight_number	
departure_airport_name	
departure_city	
arrival_airport_name	
arrival_city	
airline_name	
ticket_price	

Composite primary key:

```
(booking_id, seat_number)
```

2NF

Why the table after 1NF does not satisfy 2NF

The primary key consists of booking_id and seat_number. Not all non-key attributes depend on the full composite key: some attributes are determined only by booking_id.

Partial dependencies

```
booking_id → passenger_full_name
booking_id → passenger_passport_number
booking_id → flight_number
booking_id → departure_airport_name
booking_id → departure_city
booking_id → arrival_airport_name
booking_id → arrival_city
booking_id → airline_name
booking_id → ticket_price
```

seat_number does not participate in any of these dependencies. Therefore, the data depends only on part of the composite key.

Transformation

Seat data was moved to a separate Booking_seat table.

Booking
booking_id (PK)
passenger_full_name
passenger_passport_number
flight_number
departure_airport_name
departure_city
arrival_airport_name
arrival_city
airline_name
ticket_price

Booking_seat
booking_id (PK, FK)
seat_number (PK)

Relationship

```
Booking_seat.booking_id → Booking.booking_id
```

3NF

Why the structure after 2NF does not satisfy 3NF

The Booking table still contains non-key attributes that depend on one another. The task statement gives the following functional dependencies:

```
booking_id → flight_number
flight_number → airline_name
```

They form the following transitive dependency:

```
booking_id → flight_number → airline_name
```

Thus, airline_name is determined through another non-key attribute, flight_number, which does not satisfy 3NF.

Transformation

The data was divided into the Passenger, Airline, Airport, Flight, Booking, and Booking_seat entities.

Table	**Fields**
Passenger	passenger_id (PK), passenger_full_name, passenger_passport_number
Airline	airline_id (PK), airline_name
Airport	airport_id (PK), airport_name, city
Flight	flight_id (PK), flight_number, airline_id (FK), departure_airport_id (FK), arrival_airport_id (FK)
Booking	booking_id (PK), passenger_id (FK), flight_id (FK), ticket_price
Booking_seat	booking_id (PK, FK), seat_number (PK)

Dependencies after 3NF

```
booking_id → passenger_id
booking_id → flight_id
booking_id → ticket_price

passenger_id → passenger_full_name
passenger_id → passenger_passport_number

flight_id → flight_number
flight_id → airline_id
flight_id → departure_airport_id
flight_id → arrival_airport_id

airline_id → airline_name

departure_airport_id → departure_airport_name
departure_airport_id → departure_city

arrival_airport_id → arrival_airport_name
arrival_airport_id → arrival_city
```

Result: data for each entity is stored separately, while the relationships between tables are defined through PK and FK.

Task 3. Functional Dependencies

Dependencies used during normalization:

```
booking_id → flight_number
flight_number → airline_name
```

Dependencies of the final structure:

```
booking_id → passenger_id
booking_id → flight_id
booking_id → ticket_price

passenger_id → passenger_full_name
passenger_id → passenger_passport_number

flight_id → flight_number
flight_id → airline_id
flight_id → departure_airport_id
flight_id → arrival_airport_id

airline_id → airline_name

departure_airport_id → departure_airport_name
departure_airport_id → departure_city

arrival_airport_id → arrival_airport_name
arrival_airport_id → arrival_city
```

Task 4. Checking the Flights Table for 3NF

Primary key:

```
flight_id
```

Functional dependencies:

```
flight_id → sch_departure_time
flight_id → sch_arrival_time
flight_id → departing_airport_id
flight_id → arriving_airport_id
flight_id → departing_gate
flight_id → arriving_gate
flight_id → airline_id
flight_id → act_departure_time
flight_id → act_arrival_time
flight_id → created_at
flight_id → updated_at
```

Descriptive data about airports and the airline is not stored inside Flights. The table contains only departing_airport_id, arriving_airport_id, and airline_id.

Therefore, there are no transitive dependencies between non-key attributes inside Flights.

```
Conclusion: the Flights table satisfies 3NF.
```

Task 5. Implementation of 3NF Tables

Table creation:

```
CREATE TABLE Passenger (
passenger_id INT PRIMARY KEY,
passenger_full_name VARCHAR(100) NOT NULL,
passenger_passport_number VARCHAR(20) NOT NULL
);

CREATE TABLE Airline (
airline_id INT PRIMARY KEY,
airline_name VARCHAR(100) NOT NULL
);

CREATE TABLE Airport (
airport_id INT PRIMARY KEY,
airport_name VARCHAR(100) NOT NULL,
city VARCHAR(100) NOT NULL
);

CREATE TABLE Flight (
flight_id INT PRIMARY KEY,
flight_number VARCHAR(30) NOT NULL,
airline_id INT NOT NULL,
departure_airport_id INT NOT NULL,
arrival_airport_id INT NOT NULL,
FOREIGN KEY (airline_id) REFERENCES Airline(airline_id),
FOREIGN KEY (departure_airport_id) REFERENCES Airport(airport_id),
FOREIGN KEY (arrival_airport_id) REFERENCES Airport(airport_id)
);

CREATE TABLE Booking (
booking_id INT PRIMARY KEY,
passenger_id INT NOT NULL,
flight_id INT NOT NULL,
ticket_price DECIMAL(10,2) NOT NULL,
FOREIGN KEY (passenger_id) REFERENCES Passenger(passenger_id),
FOREIGN KEY (flight_id) REFERENCES Flight(flight_id)
);

CREATE TABLE Booking_seat (
booking_id INT NOT NULL,
seat_number VARCHAR(10) NOT NULL,
PRIMARY KEY (booking_id, seat_number),
FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
);
```

Task 5. Test Data

```
INSERT INTO Airline (airline_id, airline_name) VALUES
(1, 'Air Astana'),
(2, 'SCAT Airlines');

INSERT INTO Airport (airport_id, airport_name, city) VALUES
(1, 'Almaty International Airport', 'Almaty'),
(2, 'Astana International Airport', 'Astana'),
(3, 'Shymkent International Airport', 'Shymkent');

INSERT INTO Passenger
(passenger_id, passenger_full_name, passenger_passport_number) VALUES
(1, 'Passenger One', 'P100001'),
(2, 'Passenger Two', 'P100002'),
(3, 'Passenger Three', 'P100003'),
(4, 'Passenger Four', 'P100004'),
(5, 'Passenger Five', 'P100005');

INSERT INTO Flight
(flight_id, flight_number, airline_id, departure_airport_id, arrival_airport_id) VALUES
(1, 'KC101', 1, 1, 2),
(2, 'KC102', 1, 2, 1),
(3, 'DV701', 2, 1, 3),
(4, 'DV702', 2, 3, 1),
(5, 'KC201', 1, 1, 3);

INSERT INTO Booking
(booking_id, passenger_id, flight_id, ticket_price) VALUES
(1, 1, 1, 45000.00),
(2, 2, 2, 47000.00),
(3, 3, 3, 35000.00),
(4, 4, 4, 37000.00),
(5, 5, 5, 42000.00);

INSERT INTO Booking_seat (booking_id, seat_number) VALUES
(1, '12A'),
(1, '12B'),
(2, '15C'),
(3, '7A'),
(4, '10B'),
(5, '18F');
```

Result: the normalized structure was implemented in SQL, and the relationships between tables were defined through PK and FK.

Part 2. DDL

Task 1. Creating Tables

The tables were created with the attributes and data types specified in the task statement.

```
CREATE TABLE Airline_info (
airline_id INT,
airline_code VARCHAR(30),
airline_name VARCHAR(50),
airline_country VARCHAR(50),
created_at TIMESTAMP,
updated_at TIMESTAMP,
info VARCHAR(50)
);

CREATE TABLE Airport (
airport_id INT,
airport_name VARCHAR(50),
country VARCHAR(50),
state VARCHAR(50),
city VARCHAR(50),
created_at TIMESTAMP,
updated_at TIMESTAMP
);

CREATE TABLE Baggage_check (
baggage_check_id INT,
check_result VARCHAR(50),
created_at TIMESTAMP,
updated_at TIMESTAMP,
booking_id INT,
passenger_id INT
);

CREATE TABLE Baggage (
baggage_id INT,
weight_in_kg DECIMAL(4,2),
created_at TIMESTAMP,
updated_at TIMESTAMP,
booking_id INT
);

CREATE TABLE Boarding_pass (
boarding_pass_id INT,
booking_id INT,
seat VARCHAR(50),
boarding_time TIMESTAMP,
created_at TIMESTAMP,
updated_at TIMESTAMP
);

CREATE TABLE Booking_flight (
booking_flight_id INT,
booking_id INT,
flight_id INT,
created_at TIMESTAMP,
updated_at TIMESTAMP
);

CREATE TABLE Booking (
booking_id INT,
flight_id INT,
passenger_id INT,
booking_platform VARCHAR(50),
created_at TIMESTAMP,
updated_at TIMESTAMP,
status VARCHAR(50),
price DECIMAL(7,2)
);

CREATE TABLE Flights (
flight_id INT,
sch_departure_time TIMESTAMP,
sch_arrival_time TIMESTAMP,
departing_airport_id INT,
arriving_airport_id INT,
departing_gate VARCHAR(50),
arriving_gate VARCHAR(50),
airline_id INT,
act_departure_time TIMESTAMP,
act_arrival_time TIMESTAMP,
created_at TIMESTAMP,
```

```
updated_at TIMESTAMP
);

CREATE TABLE Passengers (
passenger_id INT,
first_name VARCHAR(50),
last_name VARCHAR(50),
date_of_birth DATE,
gender VARCHAR(50),
country_of_citizenship VARCHAR(50),
country_of_residence VARCHAR(50),
passport_number VARCHAR(20),
created_at TIMESTAMP,
updated_at TIMESTAMP
);

CREATE TABLE Security_check (
security_check_id INT,
check_result VARCHAR(20),
created_at TIMESTAMP,
updated_at TIMESTAMP,
passenger_id INT
);
```

Task 2. Defining Primary Keys

A primary key was defined for each table.

```
ALTER TABLE Airline_info ADD PRIMARY KEY (airline_id);
ALTER TABLE Airport ADD PRIMARY KEY (airport_id);
ALTER TABLE Baggage_check ADD PRIMARY KEY (baggage_check_id);
ALTER TABLE Baggage ADD PRIMARY KEY (baggage_id);
ALTER TABLE Boarding_pass ADD PRIMARY KEY (boarding_pass_id);
ALTER TABLE Booking_flight ADD PRIMARY KEY (booking_flight_id);
ALTER TABLE Booking ADD PRIMARY KEY (booking_id);
ALTER TABLE Flights ADD PRIMARY KEY (flight_id);
ALTER TABLE Passengers ADD PRIMARY KEY (passenger_id);
ALTER TABLE Security_check ADD PRIMARY KEY (security_check_id);
```

Task 4. NOT NULL Constraints

The NOT NULL constraint was applied to all non-key attributes. Primary key fields already do not allow NULL values.

```
ALTER TABLE Airline_info
ALTER COLUMN airline_code SET NOT NULL,
ALTER COLUMN airline_name SET NOT NULL,
ALTER COLUMN airline_country SET NOT NULL,
ALTER COLUMN created_at SET NOT NULL,
ALTER COLUMN updated_at SET NOT NULL,
ALTER COLUMN info SET NOT NULL;

ALTER TABLE Airport
ALTER COLUMN airport_name SET NOT NULL,
ALTER COLUMN country SET NOT NULL,
ALTER COLUMN state SET NOT NULL,
ALTER COLUMN city SET NOT NULL,
ALTER COLUMN created_at SET NOT NULL,
ALTER COLUMN updated_at SET NOT NULL;

ALTER TABLE Baggage_check
ALTER COLUMN check_result SET NOT NULL,
ALTER COLUMN created_at SET NOT NULL,
ALTER COLUMN updated_at SET NOT NULL,
ALTER COLUMN booking_id SET NOT NULL,
ALTER COLUMN passenger_id SET NOT NULL;

ALTER TABLE Baggage
ALTER COLUMN weight_in_kg SET NOT NULL,
ALTER COLUMN created_at SET NOT NULL,
ALTER COLUMN updated_at SET NOT NULL,
ALTER COLUMN booking_id SET NOT NULL;

ALTER TABLE Boarding_pass
ALTER COLUMN booking_id SET NOT NULL,
ALTER COLUMN seat SET NOT NULL,
ALTER COLUMN boarding_time SET NOT NULL,
ALTER COLUMN created_at SET NOT NULL,
ALTER COLUMN updated_at SET NOT NULL;

ALTER TABLE Booking_flight
ALTER COLUMN booking_id SET NOT NULL,
ALTER COLUMN flight_id SET NOT NULL,
ALTER COLUMN created_at SET NOT NULL,
ALTER COLUMN updated_at SET NOT NULL;

ALTER TABLE Booking
ALTER COLUMN flight_id SET NOT NULL,
ALTER COLUMN passenger_id SET NOT NULL,
ALTER COLUMN booking_platform SET NOT NULL,
ALTER COLUMN created_at SET NOT NULL,
ALTER COLUMN updated_at SET NOT NULL,
ALTER COLUMN status SET NOT NULL,
ALTER COLUMN price SET NOT NULL;

ALTER TABLE Flights
ALTER COLUMN sch_departure_time SET NOT NULL,
ALTER COLUMN sch_arrival_time SET NOT NULL,
ALTER COLUMN departing_airport_id SET NOT NULL,
ALTER COLUMN arriving_airport_id SET NOT NULL,
ALTER COLUMN departing_gate SET NOT NULL,
ALTER COLUMN arriving_gate SET NOT NULL,
ALTER COLUMN airline_id SET NOT NULL,
ALTER COLUMN act_departure_time SET NOT NULL,
ALTER COLUMN act_arrival_time SET NOT NULL,
ALTER COLUMN created_at SET NOT NULL,
ALTER COLUMN updated_at SET NOT NULL;

ALTER TABLE Passengers
ALTER COLUMN first_name SET NOT NULL,
ALTER COLUMN last_name SET NOT NULL,
ALTER COLUMN date_of_birth SET NOT NULL,
ALTER COLUMN gender SET NOT NULL,
ALTER COLUMN country_of_citizenship SET NOT NULL,
ALTER COLUMN country_of_residence SET NOT NULL,
ALTER COLUMN passport_number SET NOT NULL,
ALTER COLUMN created_at SET NOT NULL,
ALTER COLUMN updated_at SET NOT NULL;

ALTER TABLE Security_check
ALTER COLUMN check_result SET NOT NULL,
ALTER COLUMN created_at SET NOT NULL,
ALTER COLUMN updated_at SET NOT NULL,
```

```
ALTER COLUMN passenger_id SET NOT NULL;
```

Task 5. Renaming the Table

The Airline_info table was renamed to Airline.

```
ALTER TABLE Airline_info
RENAME TO Airline;
```

Task 6. Renaming the Column

In the Booking table, the price column was renamed to ticket_price.

```
ALTER TABLE Booking
RENAME COLUMN price TO ticket_price;
```

Task 7. Changing the Data Type

The data type of departing_gate in the Flights table was changed from VARCHAR(50) to TEXT.

```
ALTER TABLE Flights
ALTER COLUMN departing_gate TYPE TEXT;
```

Task 8. Dropping the Column

The info column was removed from the Airline table.

```
ALTER TABLE Airline
DROP COLUMN info;
```

Task 9. Creating Relationships Between Tables

The relationships were created using foreign keys according to the task statement.

```
ALTER TABLE Security_check
ADD CONSTRAINT fk_security_passenger
FOREIGN KEY (passenger_id)
REFERENCES Passengers(passenger_id);

ALTER TABLE Booking
ADD CONSTRAINT fk_booking_passenger
FOREIGN KEY (passenger_id)
REFERENCES Passengers(passenger_id);

ALTER TABLE Baggage_check
ADD CONSTRAINT fk_baggage_check_passenger
FOREIGN KEY (passenger_id)
REFERENCES Passengers(passenger_id);

ALTER TABLE Baggage_check
ADD CONSTRAINT fk_baggage_check_booking
FOREIGN KEY (booking_id)
REFERENCES Booking(booking_id);

ALTER TABLE Baggage
ADD CONSTRAINT fk_baggage_booking
FOREIGN KEY (booking_id)
REFERENCES Booking(booking_id);

ALTER TABLE Boarding_pass
ADD CONSTRAINT fk_boarding_pass_booking
FOREIGN KEY (booking_id)
REFERENCES Booking(booking_id);

ALTER TABLE Booking_flight
ADD CONSTRAINT fk_booking_flight_booking
FOREIGN KEY (booking_id)
REFERENCES Booking(booking_id);

ALTER TABLE Booking_flight
ADD CONSTRAINT fk_booking_flight_flight
FOREIGN KEY (flight_id)
REFERENCES Flights(flight_id);

ALTER TABLE Flights
ADD CONSTRAINT fk_flights_departure_airport
FOREIGN KEY (departing_airport_id)
REFERENCES Airport(airport_id);

ALTER TABLE Flights
ADD CONSTRAINT fk_flights_arrival_airport
FOREIGN KEY (arriving_airport_id)
REFERENCES Airport(airport_id);

ALTER TABLE Flights
ADD CONSTRAINT fk_flights_airline
FOREIGN KEY (airline_id)
REFERENCES Airline(airline_id);
```

Result: all tables were created, primary and foreign keys were defined, required fields were constrained with NOT NULL, and all required structural changes were completed.